Team Vyflare Synthetic Vial Image Generation with the CSI Digital Twin

Engineering Technical Report

NVIDIA Rockwell Physical AI for the Factory Floor Hackathon

Report item	Details
Prepared for	Team Vyflare and Amy's vial image research
Platform	NVIDIA Brev with Isaac Sim 6.0.1
Digital twin	CSI fill station and FillCam
Report date	26 October 2026
Data status	Synthetic experiments with separate real data evaluation

We built and verified a non-destructive rendering workflow that varies authored animation frames and liquid display colors while preserving the supplied digital twin. The completed full-quality corpus contains 240 traceable images. These images are suitable for controlled synthetic-data experiments, not for assigning recipes, measured volume, real mixture ratios, quality states, or defects.

Contents

1 Executive Summary

2 Scope and Evidence Boundaries

3 Digital Twin Architecture

4 Fill Station Animation and FillCam

5 Baseline Image Rendering

6 Non Destructive Synthetic Generation

7 Dataset Design and Results

8 Metadata and Traceability

9 Verification and Source Protection

10 Reproduction on NVIDIA Brev

11 Use in Vial Image Model Development

12 Limitations and Risks

13 Recommended Next Steps

Appendix A Project Paths

Appendix B Experiment Parameters

1 Executive Summary

Team Vyflare used the CSI fill-station digital twin in Isaac Sim 6.0.1 to create a controlled synthetic vial-image pipeline for Amy's research. We first inspected the stage composition, authored animation, FillCam configuration, and the supplied example renderer. We then added a separate experiment manifest and renderer that apply temporary material overrides in an anonymous USD session layer. The workflow never saves those overrides into the source USD or USDC assets.

The completed full-quality output contains 240 images across three datasets: a 65-image baseline sweep, a 115-image temporal sweep, and a 60-image color sweep. Two six-image smoke runs were also used to check the execution path and are reported separately. Every full-quality image has a metadata record with its animation frame, display color, camera and render settings, material attributes changed, source-file hashes, and image hash. Automated checks confirmed image counts, parameter coverage, metadata labels, hash integrity, and unchanged protected source files.

The synthetic data can support experiments in vial or liquid localization, broad color discrimination, and recognition of visual states along the authored animation. It does not establish real fill volume, recipe identity, mixture ratio, defect status, or process carryover. Model value must therefore be measured on held-out real vials or acquisition sessions, with the synthetic data used as pretraining or controlled augmentation rather than as the final source of truth.

2 Scope and Evidence Boundaries

This report records work completed during the digital-twin image-generation workflow and the evidence used to verify it.

Status	Meaning in this report
Confirmed	Observed in the project files, generated outputs, run metadata, or verification logs.
Interpretation	Engineering judgment based on confirmed behavior, stated with its limits.
Recommendation	Proposed follow-up work that was not part of the completed run.

The digital-twin work changed only experiment files and generated output directories. The supplied source assets remained intact. The real vial dataset was not uploaded, altered, relabeled, or analyzed as part of this workflow. A later comparison against real data should preserve that separation and use held-out real sessions for evaluation.

Color names such as cyan, amber, green, and violet describe render settings only. They do not correspond to a recipe, real chemical, material identity, mixture ratio, quality decision, or defect. Labels such as early, middle, and final describe locations in the authored animation and are not calibrated fill percentages.

3 Digital Twin Architecture

The fill-station scene is split across human-readable USD stage files and binary geometry. The stage file authors composition and visibility timing, while the binary geometry layer contains the detailed meshes, transforms, material bindings, camera, and lights. This separation explains why the selected Fill_liquid parent can show no direct material in the Isaac Sim interface even though visible descendant geometry is shaded.

Component	Role
csi_line.usda	Top-level CSI line scene and station composition.
stations/fill_station/fill_station.usda	Fill-station stage, sublayer reference, and visibility time samples.
stations/fill_station/geometry/fill_station.usdc	Detailed geometry, transform animation, descendant material bindings, FillCam, and lights.
cameras/FillCam.json	Exported FillCam calibration, image size, camera paths, frame range, and frame rate.
tools/render_example.py	Reference one-frame renderer supplied with the digital twin.

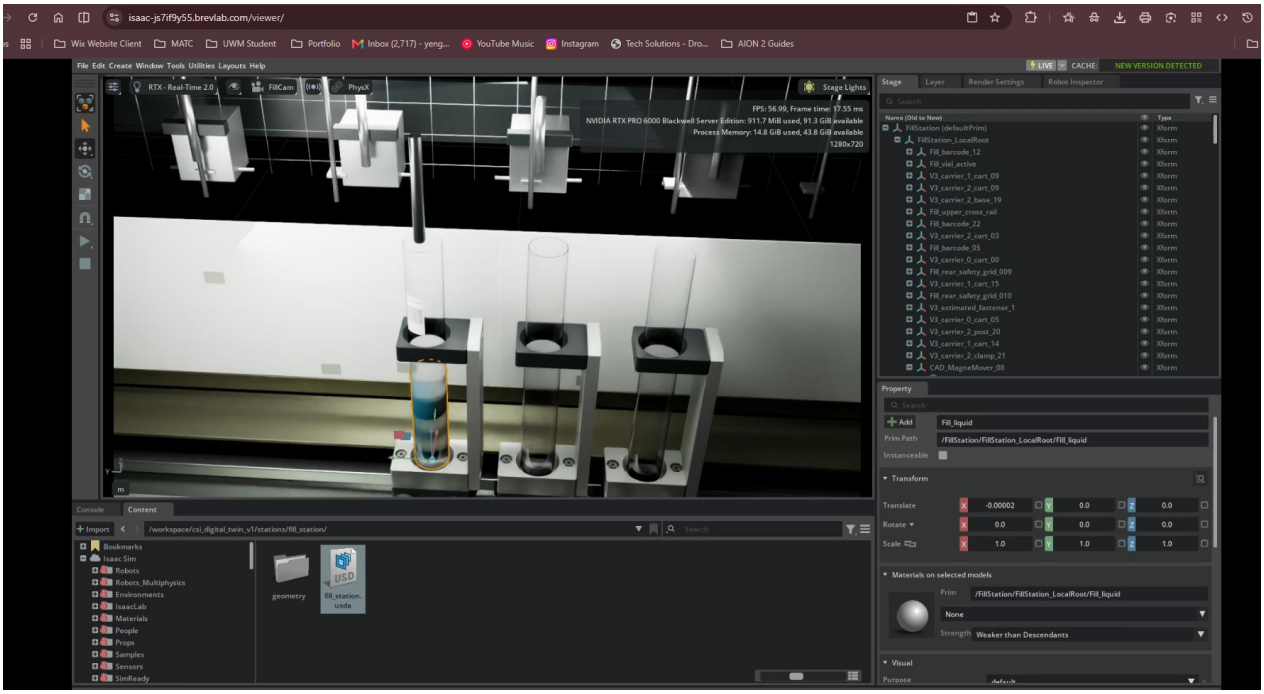


Figure 1 Fill station and selected Fill_liquid parent in Isaac Sim

4 Fill Station Animation and FillCam

4.1 Authored fill station animation

The station uses a prerecorded authored animation spanning frames 1 through 686 at 30 frames per second. The rendering workflow selects a time code; it does not simulate an active filling process or solve fluid physics. The visual fill state is therefore the appearance stored at the selected frame.

Animated object	Visibility time samples
Fill_dispensing_stream	Invisible at 1; visible at 12; invisible at 368; visible at 586; invisible at 661
Fill_liquid	Invisible at 1; visible at 21
Fill_vial_neighbour_2	Invisible at 1; visible at 181

These samples control whether scene elements participate in the authored sequence. Detailed transforms and material bindings are stored in the geometry layer. Because no volume calibration is present, frame number must not be converted into milliliters or a fill percentage without an independent calibration procedure.

4.2 FillCam

FillCam is the fixed inspection camera used for all reported images. Its exported configuration describes a 640 by 480 pixel pinhole camera with no lens distortion. The focal length is 40.993 mm, the aperture is 36 by 27 mm, and the focal length in pixel units is approximately 728.77.

Parameter	Value
Station camera prim	/FillStation/FillStation_LocalRoot/FillCam/FillCam
Full line camera prim	/CSI_Line/FillStation/FillStation_LocalRoot/FillCam/FillCam
Resolution	640 x 480 pixels
Projection	Pinhole
Lens distortion	None, k1 = 0
Animation and rate	Frames 1 to 686 at 30 fps

5 Baseline Image Rendering

The supplied tools/render_example.py starts Isaac Sim in headless mode with path tracing, opens a USD stage, selects the requested camera prim, and sets the stage time from the frame and timeCodesPerSecond values. It creates a Replicator render product, attaches an RGB annotator, advances the orchestrator, and saves the image.

1. Launch Isaac Sim headlessly with the requested renderer.
2. Open the stage and resolve the camera prim.
3. Set the authored frame as the current USD time code.
4. Create the render product and attach the RGB annotator.
5. Advance rendering for the configured subframes and write the RGB image.

The example renderer also applies an anonymous-session clip fix and can enable lens postprocessing for the CapSeal camera. For FillCam, the exported camera configuration specifies a direct 640 by 480 pinhole render.

6 Non Destructive Synthetic Generation

We added tools/render_synthetic_experiments.py and a JSON experiment manifest. The runner keeps one Isaac Sim session open, loads the fill-station stage, discovers the materials bound below Fill_liquid and Fill_dispensing_stream, converts each requested sRGB color to linear RGB, and authors the shader inputs:diffuseColor overrides in the anonymous USD session layer.

The runner never calls Save or SaveSessionLayers. It writes PNG and metadata files to a separate output directory. Before and after rendering, it hashes the protected stage, geometry, camera, renderer, and configuration files. A source change causes the run to fail rather than silently modifying the digital twin.

Original asset behavior	Experiment behavior
USD and USDC source files remain the source of geometry, animation, camera, and lighting.	Temporary material attributes are authored only in the anonymous session layer.
Authored frame controls the visible animation state.	The manifest selects frames but assigns no measured fill volume.
Existing descendant material bindings remain in place.	The runner changes only discovered shader diffuse-color inputs.
Project files remain in their original locations.	Images and metadata are written under synthetic_outputs.

7 Dataset Design and Results

The experiment was expanded in stages. A small smoke sweep first validated the full path. Three full-quality datasets then increased temporal coverage and display-color coverage while retaining the same FillCam view, 640 by 480 output, and 32 path-tracing subframes.

Dataset	Design	Images	Purpose
Baseline sweep v2	13 frames x 5 colors	65	Balanced frame and color coverage
Temporal sweep v3	23 frames x 5 colors	115	Denser coverage from frame 30 through 686
Color sweep v4	5 frames x 12 colors	60	Broader display-color coverage at selected states
Full-quality total	Three verified datasets	240	Primary synthetic corpus

The smoke test produced six images from frames 90, 301, and 686 crossed with cyan and amber. It was run twice in separate output directories to confirm repeatable execution. Smoke images are operational checks and are not included in the 240-image full-quality total.

7.1 Frame and color coverage

Dataset	Frames	Display colors
Baseline sweep v2	30, 60, 90, 150, 210, 270, 330, 390, 480, 540, 600, 630, 686	Blue, cyan, green, amber, magenta
Temporal sweep v3	Every 30 frames from 30 through 660, plus 686	Blue, cyan, green, amber, magenta
Color sweep v4	90, 210, 390, 600, 686	Navy, blue, cyan, teal, green, lime, yellow, amber, orange, red, magenta, violet

8 Metadata and Traceability

Each output directory contains the rendered PNG files, metadata.jsonl, metadata.csv, and run_manifest.json. The per-image rows make each image traceable to its render settings. The run manifest records the shared execution context and source-file integrity results.

The interpretation fields are part of the data contract. They prevent a downstream pipeline from treating a display color as a recipe or a selected frame as a calibrated physical measurement.

Metadata group	Recorded values
Identity	Experiment ID, case ID, image path, and image SHA-256
Animation	Frame, time, and authored-state label
Color	Display-color name, sRGB hex, applied linear RGB, and exact shader attributes
Imaging	FillCam prim, 640 x 480 resolution, path-tracing mode, and subframes
Provenance	Source paths, source SHA-256 values, and session-layer status
Interpretation	Synthetic-experiment label, no measured fill volume, no source save, and other changes

9 Verification and Source Protection

Automated verification checked exact image counts, unique case identifiers, expected frame and color coverage, 32 subframes, matching PNG SHA-256 values, required synthetic labels, manifest image counts, anonymous-session use, and unchanged protected source files. Representative images were also inspected visually.

Run	Verified result
Baseline sweep v2	65 images, 13 frames x 5 colors, metadata and hashes valid, protected sources unchanged
Temporal and color sweeps	175 new images: 115 temporal and 60 color, metadata and hashes valid, protected sources unchanged
Combined full-quality corpus	240 images across the three datasets

Visual checks confirmed that images at a shared frame preserved scene geometry while liquid appearance changed with the display-color setting. Checks across frames confirmed that the existing authored animation changed the visible scene state. These observations support renderer correctness but do not supply physical or process labels.

10 Reproduction on NVIDIA Brev

Use a Brev instance with an NVIDIA RTX GPU, compatible drivers, and Isaac Sim 6.0.1. Keep the internal project layout unchanged. The paths below reflect the verified Brev workspace used for this work.

10.1 Upload and inspect

```
cd /workspace/csi_digital_twin_v1
ls tools/render_synthetic_experiments.py experiments/synthetic_liquid_sweep_v1.json
```

10.2 Run the reproducible smoke test

```
/path/to/isaac-sim/python.sh tools/render_synthetic_experiments.py \
  --config experiments/synthetic_liquid_sweep_v1.json \
  --subframes 8 \
  --out-dir /workspace/synthetic_outputs/smoke_test
```

For the manifest's full-quality setting, omit --subframes 8 or set --subframes 32 and use a new output directory.

10.3 Verify the output

```
cd /workspace/synthetic_outputs/smoke_test
ls -1 *.png | wc -l
ls -l metadata.jsonl metadata.csv run_manifest.json
```

Confirm six PNG files for the smoke manifest. Inspect metadata.jsonl for data_status set to synthetic_experiment, fill_is_measured_volume set to false, source_files_saved set to false, and an image hash for every record. Then compare the source hashes in run_manifest.json with the recorded post-run values.

For the completed datasets, the verified output directories were /workspace/synthetic_outputs/sweep_v2_65_subframes32,

/workspace/synthetic_outputs/temporal_v3_115_subframes32, and /workspace/synthetic_outputs/color_v4_60_subframes32.

11 Use in Vial Image Model Development

The synthetic corpus is most useful when it increases controlled visual coverage that is scarce in the real training set. It can support pretraining or augmentation for vial and liquid localization, broad display-color discrimination, and recognition of scene appearances along the authored animation.

A credible evaluation should compare a real-only baseline with synthetic pretraining followed by real fine-tuning, and with a mixed training set that contains a limited synthetic fraction such as 25 to 50 percent. Hyperparameters and model selection should use real validation data. Final metrics should come from untouched real vials or entire held-out acquisition sessions to reduce leakage from near-duplicate images.

Task	Suggested evaluation on held-out real data
Vial or liquid detection	Precision, recall, and mean average precision
Color classification	Accuracy or a color-distance metric using independently confirmed real labels
Fill boundary estimation	Boundary error in pixels, or physical units only after real calibration
Robustness	Performance by real session, vial, lighting condition, and camera condition

A useful result is an improvement on held-out real data, not a high score on synthetic images alone. If synthetic training improves synthetic validation but not real validation, the likely issue is domain gap or label mismatch.

12 Limitations and Risks

- The selected frame is an authored visual state, not a measured liquid volume or calibrated fill percentage.
- Display colors have no recipe, chemistry, mixture-ratio, quality, or defect meaning.
- Images cannot establish carryover or another process root cause without real process evidence.
- The current sweeps hold camera, background, and most lighting fixed, so visual diversity remains narrower than real operations.
- The current runner changes frame and liquid diffuse color. It does not yet vary camera pose, illumination, reflections, focus, motion blur, sensor noise, or vial placement.
- Synthetic metadata is reliable for render parameters but does not replace independently collected real labels.

13 Recommended Next Steps

1. Create a real-data inventory without modifying the source dataset. Record sessions, vials, camera setup, image dimensions, and the availability of trusted labels.
2. Measure the real image distribution before extending the generator. Use observed ranges for lighting, camera pose, reflections, focus, noise, occlusion, and vial placement.
3. Extend the manifest and metadata schema to record each new domain-randomization parameter. Preserve the same source-hash and session-layer safeguards.
4. Define a session-aware train, validation, and test split in which all images from one real vial or acquisition session stay in one split.
5. Run the real-only, synthetic-pretrain, and mixed-training comparisons and report results by held-out real session.
6. Review failure cases with Amy and refine synthetic coverage only where a documented real-data gap exists.

Appendix A Project Paths

Item	Path
Local project	C:\Users\Yengtha\Downloads\nvidia ai hackathon\csi_digital_twin_v1
Brev project	/workspace/csi_digital_twin_v1
Synthetic runner	tools/render_synthetic_experiments.py
Initial manifest	experiments/synthetic_liquid_sweep_v1.json
Run guide	SYNTHETIC_EXPERIMENT.md
Brev outputs	/workspace/synthetic_outputs

Appendix B Experiment Parameters

Color name	sRGB hex
Navy	#2446A8
Blue	#2078D9
Cyan	#20C7D9
Teal	#159A9C
Green	#35B779
Lime	#8BC34A
Yellow	#F4D03F
Amber	#F2A51A
Orange	#E76F2E
Red	#D94A4A
Magenta	#C64BD8
Violet	#7656C9

All hexadecimal values in this appendix are synthetic display parameters. The renderer converts them from sRGB to linear RGB before applying the shader override and records both forms in metadata.